LAB REPORT-5

Dhruv Bansal(2023102048, Group 6, Table No:7)

EXPERIMENT-1

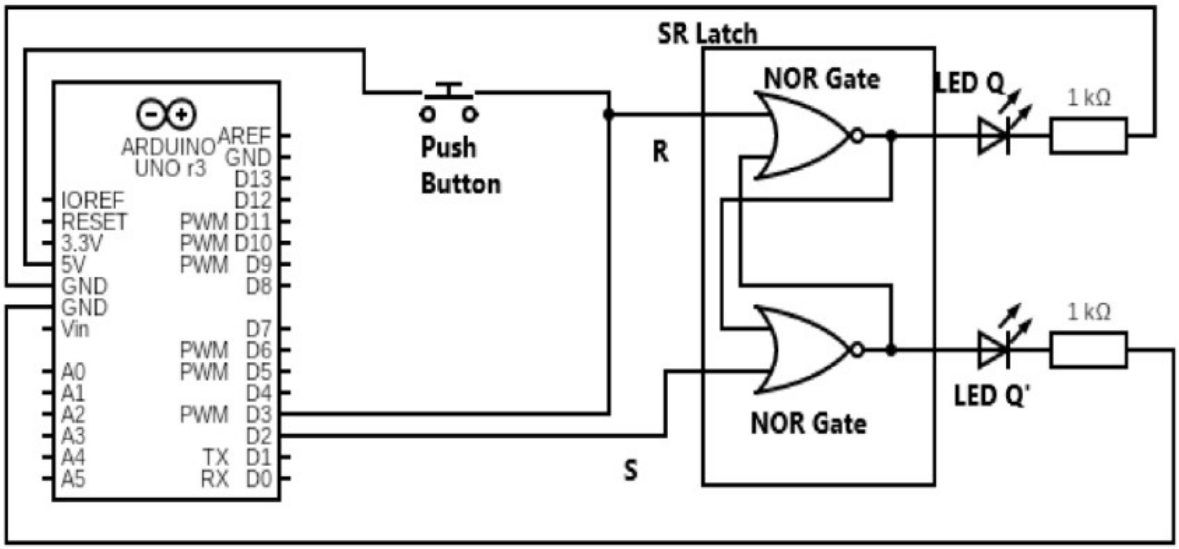
Objective:

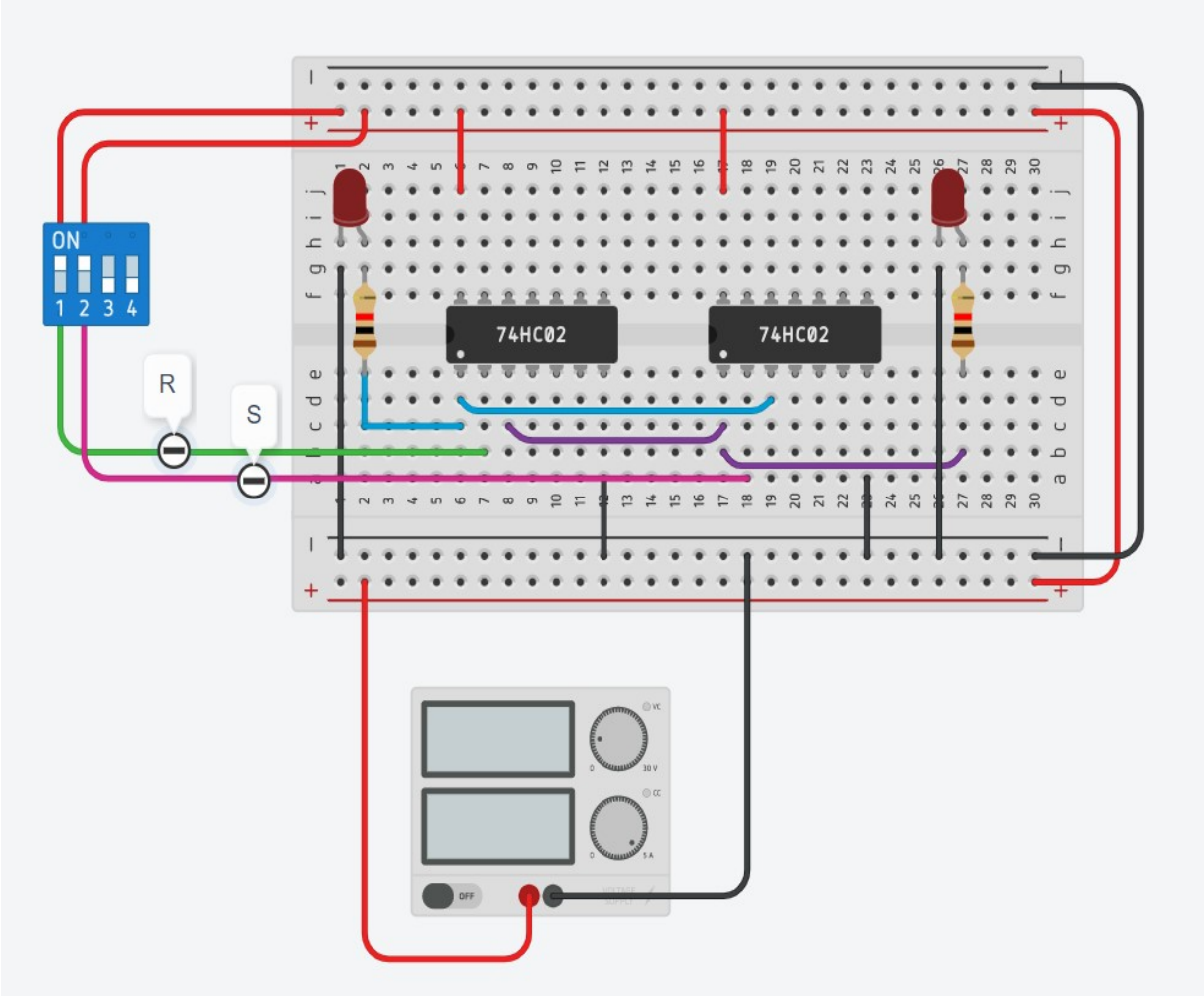
To design, assemble and test a SR latch using NOR gates.

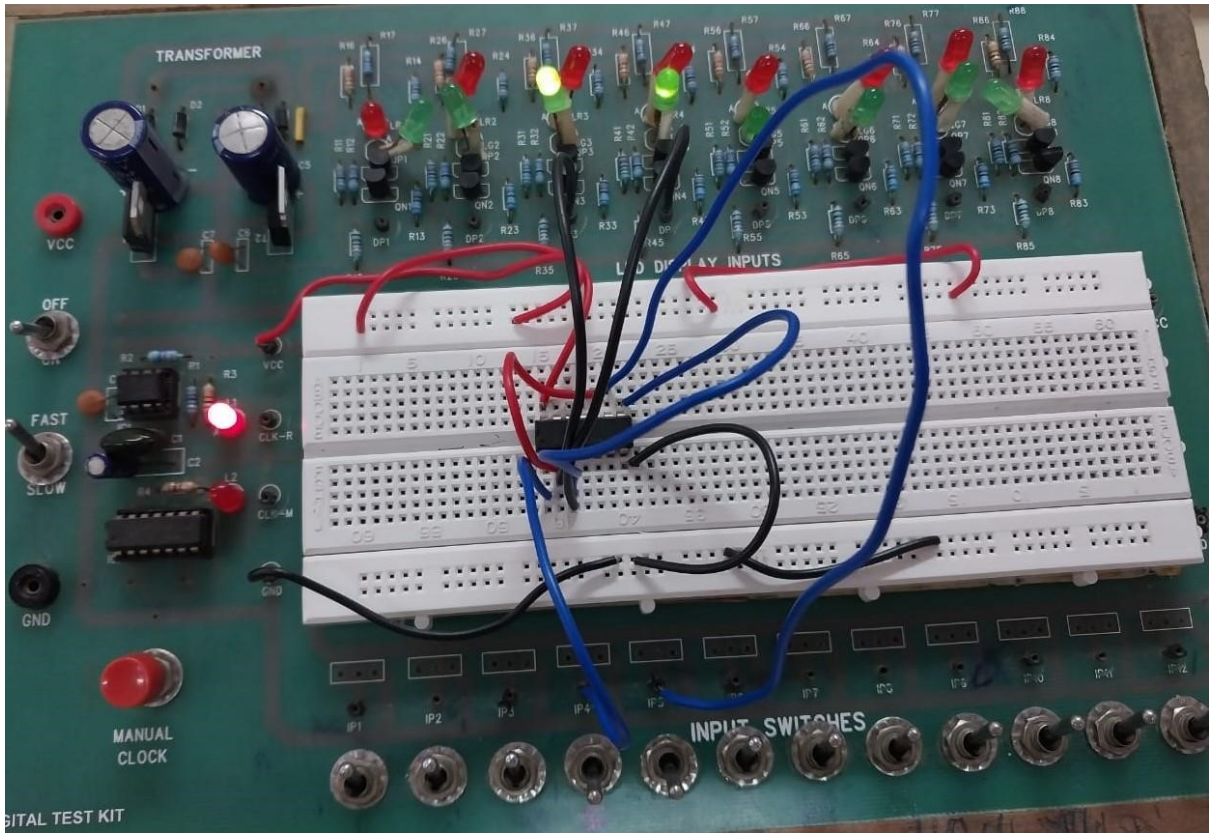
Electric Components Required

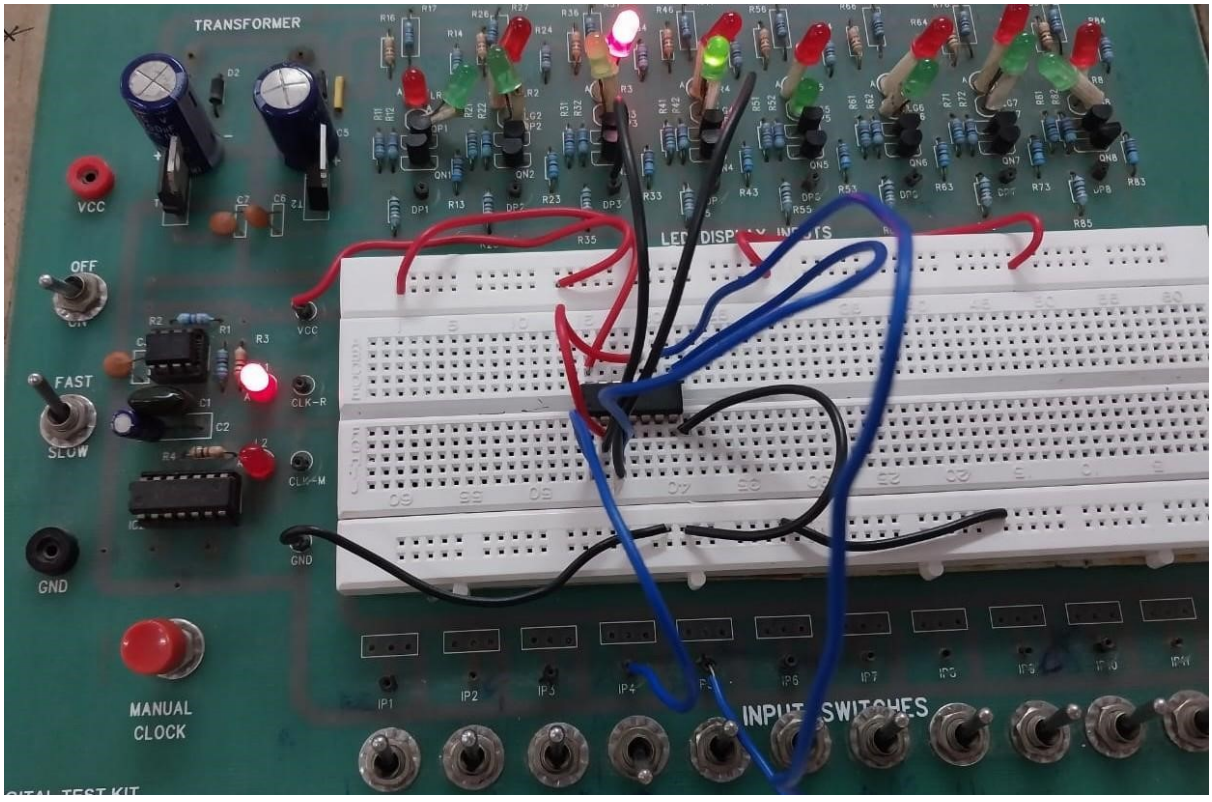
1. *The Digital Test Kit*
2. *Wires to connect pins in the Digital Testkit*
3. *1 CD4001 IC*

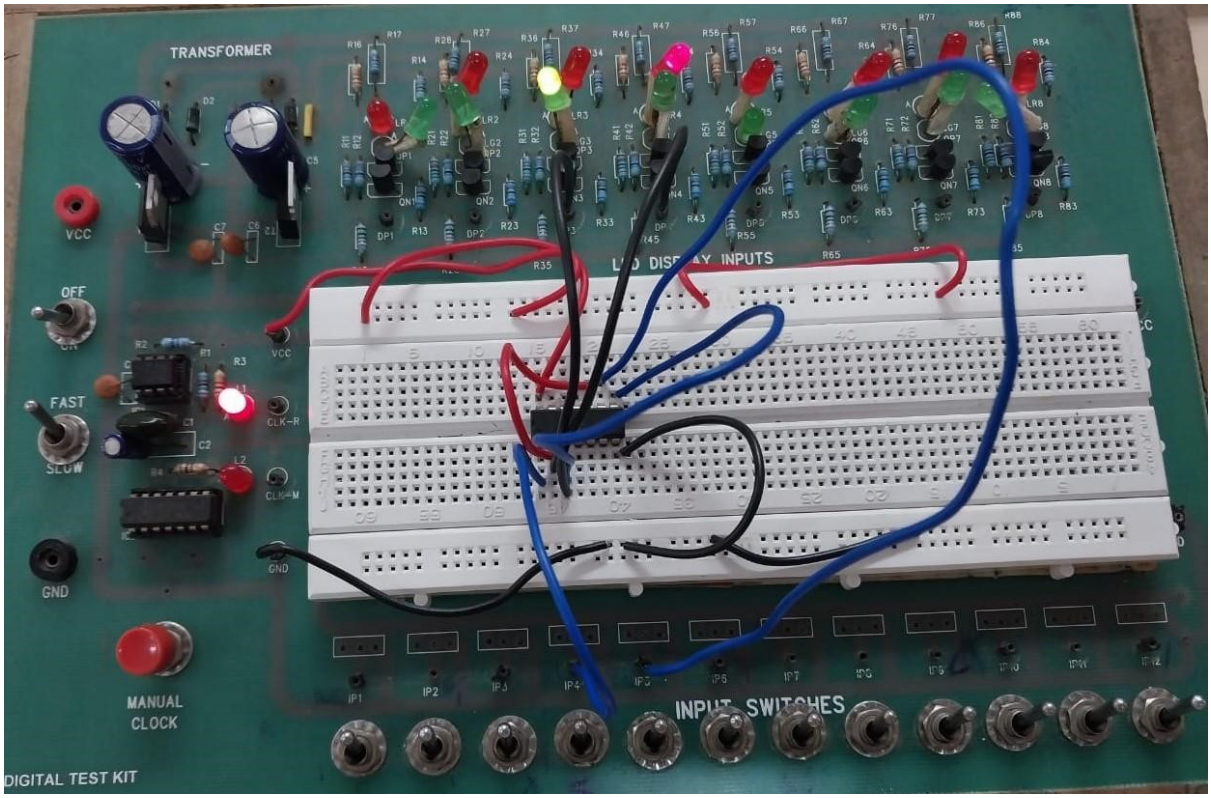
The Reference Circuit




ON
1 2 3 4
R
S
74HC02
74HC02
OFF


TRANSFORMER
VCC
OFF
FAST
SLOW
GND
CLK-R
CLK-M
MANUAL
CLOCK
INPUT SWITCHES
IP1
IP2
IP3
IP4
IP5
IP6
IP7
IP8
IP9
IP10
IP11
IP12
GITAL TEST KIT


TRANSFORMER
VCC
OFF
FAST
SLOW
GND
CLK-R
CLK-M
MANUAL
CLOCK
INPUT SWITCHES
IP1
IP2
IP3
IP4
IP6
IP7
IP8
IP9
IP10
IP11
IP12



Procedure:

Steps to be followed to perform the experiment are:

1. *Connect the VCC and GND pins on the Digital Test Kit to the VCC and GND lines on the top and bottom of the breadboard using RED and BLACK wires respectively.*
2. *Connect the VCC and GND pins of the IC to the VCC and GND lines on the top and bottom of the breadboard.*
3. *Verify that all the gates present in the IC are working.*
4. *Connect one of the input pins of any gate in the CD4001 IC to one of the IP1-IP12 Input switches i.e, R*
5. *Connect one of the input pins of another gate in the same IC to one of the IP1-IP12 Input switches i.e, S*
6. *Now, Cross the outputs of both gates as second input for each other as shown in the Reference Circuit diagram.*

7. *Connect the corresponding output pins of the gates to any two of the DP1-DP8 LED display points i.e, Q and Q' provided in the Test Kit.*
8. *Now, Switch on the VCC supply and put the Clock control switch in the FAST position to avoid breakage of ICs.*
9. *Apply the possible combinations of R and S to the gate inputs one by one and tabulate the corresponding values of the gate output (Q and Q') as observed on the LED displays.*
10. *Interpret the observed result and verify the latch operation.*

Observation:

Observed Outputs/Results:

S	R	Q	Q’
0	1	0	1
0	0	0	1
1	0	1	0
0	0	1	0
0	1	0	1
1	0	1	0
0	1	0	1
0	0	0	1
1	1	0	0
0	0	0	1
1	0	1	0
1	1	0	0
0	0	1	0
0	1	0	1
1	1	0	0
0	0	1	0

Conclusion:

1. SR latch is designed and assembled successfully and it’s working has been verified.

Link to the Tinkercad Simulation:

LAB 5 RS

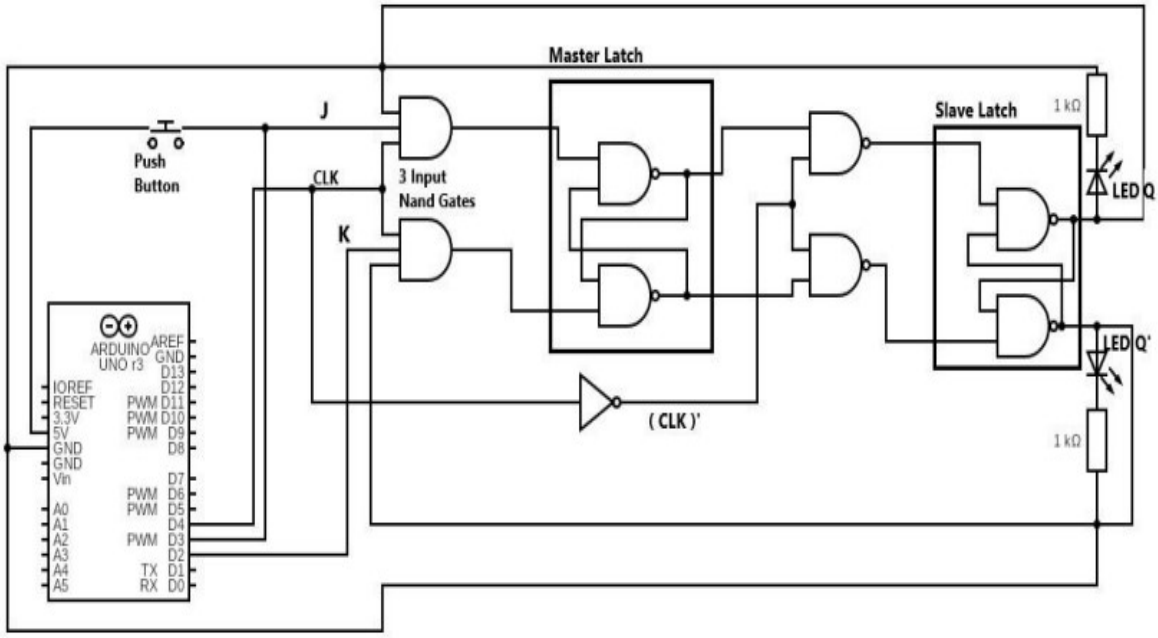
EXPERIMENT-2

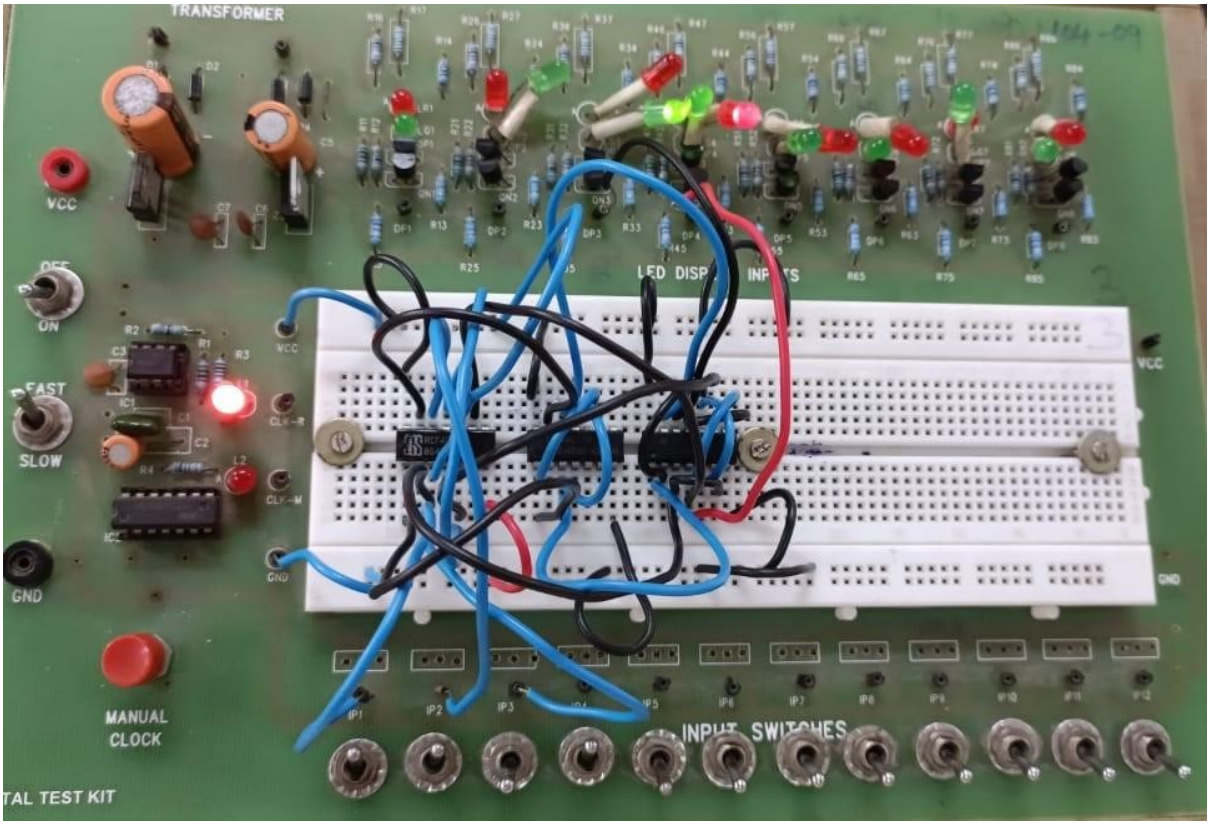
Objective:

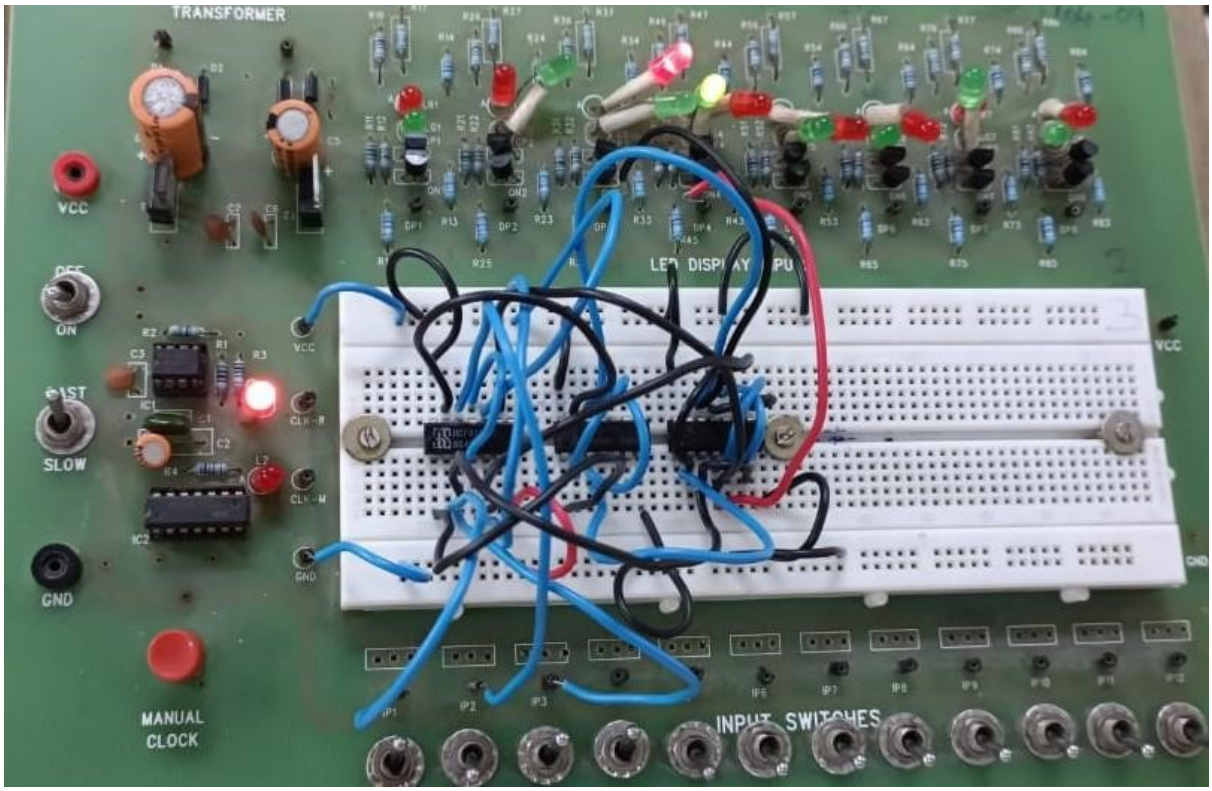
To design, assemble and test a JK Master Slave Flip Flop.

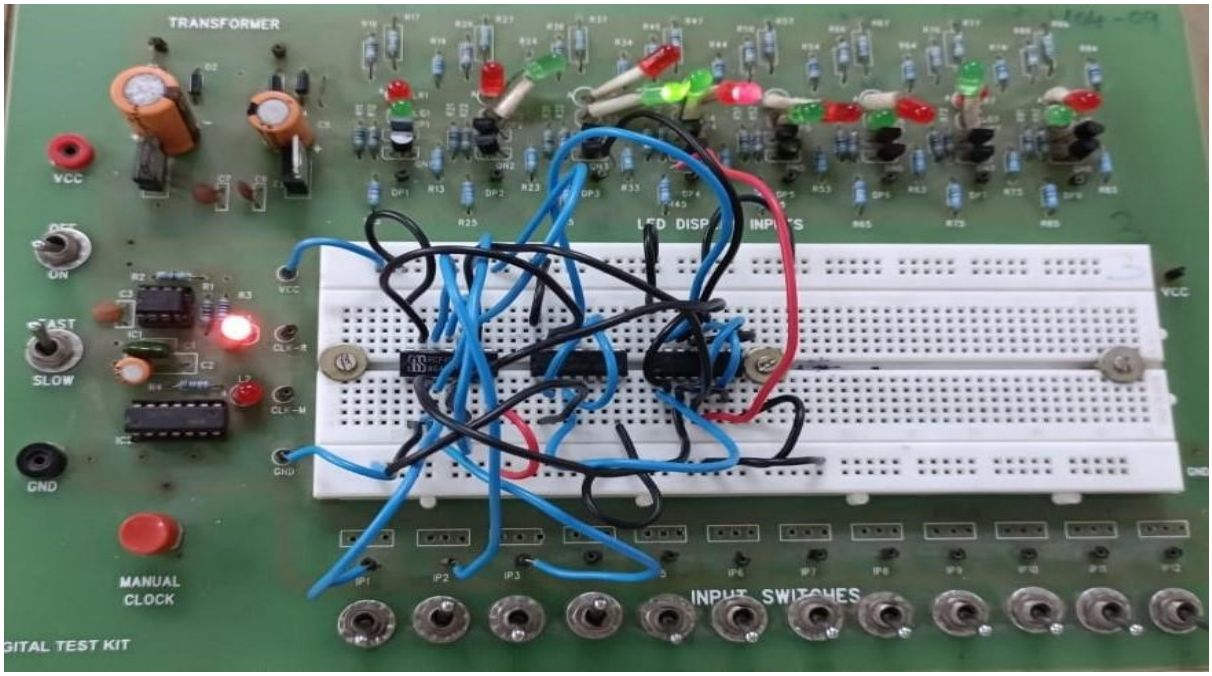
Electric Components Required:

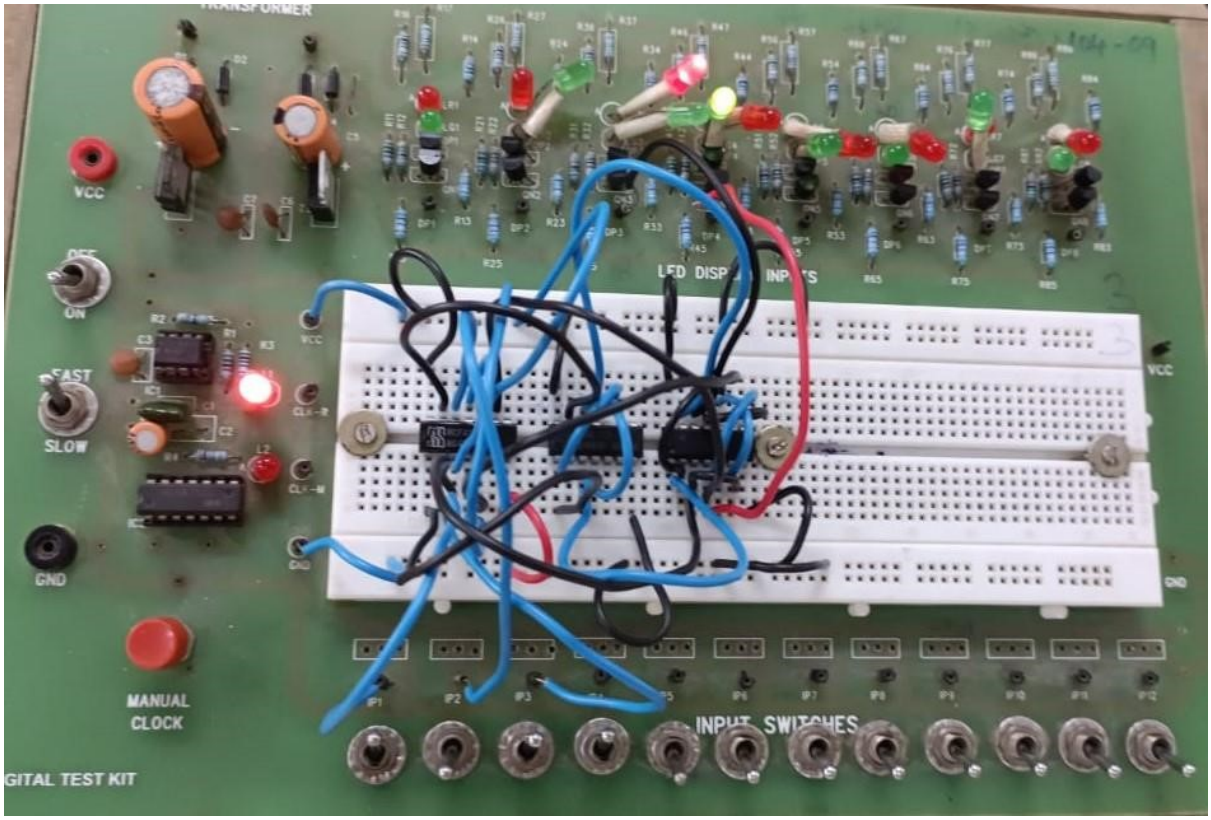
1. *The Digital Test Kit*
2. *Wires to connect pins in the Digital Test Kit*
3. *1 CD4012 IC (4-input NAND)*
4. *1 CD4011 IC (2-input NAND)*
5. *1 CD4001 IC (2-input NOR)*


Master Latch
Slave Latch
J
K
CLK
Push
Button
3 Input
Nand Gates
1 kΩ
LED Q
LED Q'
1 kΩ
(CLK)'
ARDUINO
UNO r3
AREF
GND
D13
D12
IOREF
RESET
3.3V
5V
GND
GND
Vin
A0
A1
A2
A3
A4
A5
PWM D11
PWM D10
PWM D9
D8
D7
PWM D6
PWM D5
PWM D3
D2
TX D1
RX D0


TRANSFORMER
VCC
OFF
ON
FAST
SLOW
GND
MANUAL
CLOCK
INPUT SWITCHES
TAL TEST KIT


TRANSFORMER
VCC
ON
SLOW
GND
MANUAL
CLOCK
INPUT SWITCHES


TRANSFORMER
VCC
OFF
ON
FAST
SLOW
GND
MANUAL
CLOCK
INPUT SWITCHES
VCC
GND


VCC
OFF
ON
FAST
SLOW
GND
MANUAL
CLOCK
VCC
GND
INPUT SWITCHES
GITAL TEST KIT

Procedure:

Steps to be followed to perform the experiment are:

1. *Connect the VCC and GND pins on the Digital Test Kit to the VCC and GND lines on the top and bottom of the breadboard using RED and BLACK wires respectively.*
2. *Connect the VCC and GND pins of the ICs to the VCC and GND lines on the top and bottom of the breadboard.*
3. *Take J, K and CK from any three of the IP1-IP12 Input switches*
4. *Assemble the circuit as shown in the Reference Circuit diagram by using two 4-input NAND gates from CD4012 chip as 3-input gates.*
5. *Connect the unused inputs of the 4-input NAND gates to the VCC line on the top of the breadboard.*
6. *Connect the outputs Q and Q' to any two of the DP1-DP8 LED display points.*
7. *Now, Switch on the VCC supply and put the Clock control switch in the FAST position to avoid breakage of ICs.*
8. *Apply the possible combinations of J and K to the gate inputs one by one and tabulate the corresponding values of the gate outputs (Q and Q') as observed on the LED displays.*
9. *Interpret the observed result and verify the latch operation.*

Observation:

Observed outputs/Results:

J	*K*	*ACTION*	Q_{n+1}
0	*0*	*HOLD*	Q_n

0	1	CLEAR	0
1	0	SET	1
1	1	TOGGLE	Q_n'

Conclusion:

1. The circuit realises the required logic for the R-S inputs of the Master and Slave latches constituting a J-K MasterSlave flip flop.

$$R_M = K.Q.CK$$
$$S_M = J.Q'.CK$$
$$S_S = Q_M \, . \, CK'$$
$$R_S = Q_M'. \, CK'$$

2. JK Master-Slave flip flop is designed and assembled successfully and it's working has been verified.

Link to the Tinkercad Simulation:

JK FLIPFLOP

PART-C: (of lab 5)4-BIT UP-DOWN COUNTER

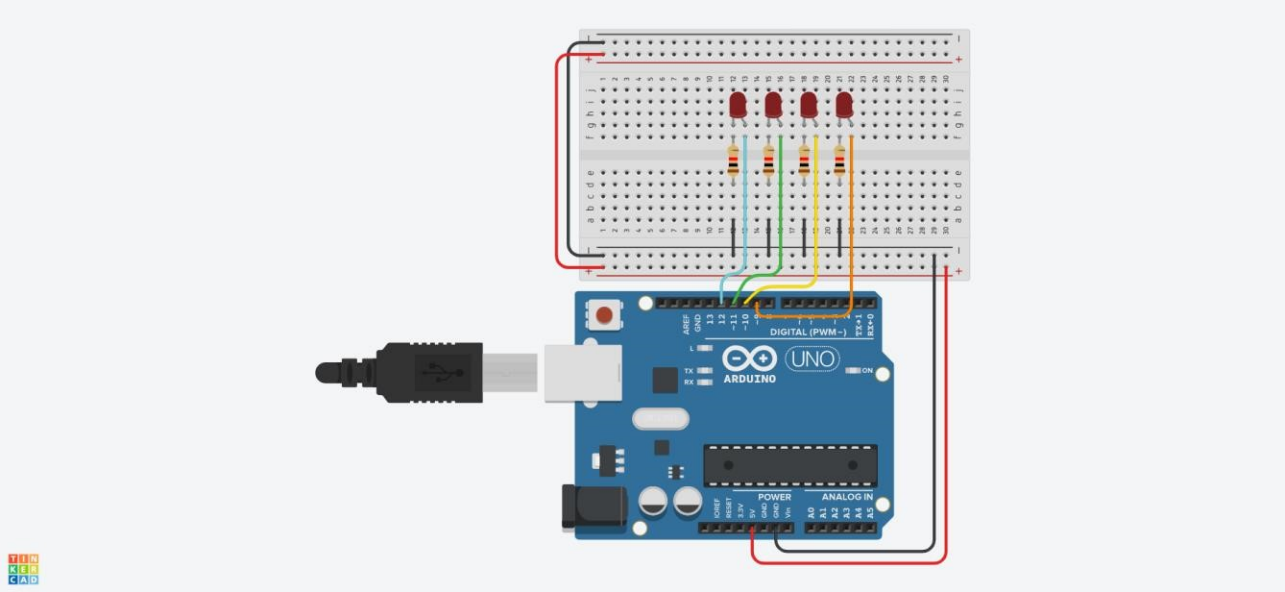
OBJECTIVE :

We aim to create a 4 bit counter which displays numbers from 0 to 15 and back to 0 in binary.

ELECTRONIC COMPONENTS REQUIRED :

Arduino, Breadboard, LEDs, Resistors, Wires.

THE REFERENCE CIRCUIT :



PROCEDURE :

1. *On a bread board, make the connections as shown above in the referennce circuit.*
2. *Properly write the code to Arduino which is given below.*
3. *Take care of the pin connections.*
4. *Simulate the circuit made.*
5. *As time.h is not included in some compilers, we need to write the complete code to the header file also.*

CODE TO THE ARDUINO :

```
#ifndef Event_h
#define Event_h

#include <inttypes.h>

#define EVENT_NONE 0
#define EVENT_EVERY 1
#define EVENT_OSCILLATE 2

class Event
{

public:
  Event(void);
  void update(void);
  void update(unsigned long now);
  int8_t eventType;
  unsigned long period;
  int repeatCount;
  uint8_t pin;
  uint8_t pinState;
  void (*callback)(void);
  unsigned long lastEventTime;
  int count;
};
```

```
#endif

#ifndef Timer_h
#define Timer_h

#include <inttypes.h>


#define MAX_NUMBER_OF_EVENTS (10)

#define TIMER_NOT_AN_EVENT (-2)
#define NO_TIMER_AVAILABLE (-1)

class Timer
{

public:
  Timer(void);

  int8_t every(unsigned long period, void (*callback)(void));
  int8_t every(unsigned long period, void (*callback)(void), int
repeatCount);
  int8_t after(unsigned long duration, void (*callback)(void));
  int8_t oscillate(uint8_t pin, unsigned long period, uint8_t
startingValue);
  int8_t oscillate(uint8_t pin, unsigned long period, uint8_t
startingValue, int repeatCount);
```

```
  /**
   * This method will generate a pulse of !startingValue, occuring period after the
   * call of this method and lasting for period. The Pin will be left in !startingValue.
   */
  int8_t pulse(uint8_t pin, unsigned long period, uint8_t startingValue);

  /**
   * This method will generate a pulse of pulseValue, starting immediately and of
   * length period. The pin will be left in the !pulseValue state
   */
  int8_t pulseImmediate(uint8_t pin, unsigned long period, uint8_t pulseValue);
  void stop(int8_t id);
  void update(void);
  void update(unsigned long now);

protected:
  Event _events[MAX_NUMBER_OF_EVENTS];
  int8_t findFreeEventIndex(void);

};
#endif
```

```
//// YOUR CODE STARTS HERE

Timer t;

int i1 = 9;
int i2 = 10;
int i3 = 11;
int i4 = 12;
int state = 0;

void setup() {

  Serial.begin(9600);

  pinMode(i1, OUTPUT);
  pinMode(i2, OUTPUT);
  pinMode(i3, OUTPUT);
  pinMode(i4, OUTPUT);

  t.oscillate(i1, 1000, 0, 8);
  t.oscillate(i2, 2000, 0, 4);
  t.oscillate(i3, 4000, 0, 2);
  t.oscillate(i4, 8000, 0, 1);

  t.every(16000, reverseTimers);

}
```

```
// 1 unit of your timer = 500ms in real time

void loop() {

  t.update();

}

// "every" X milliseconds
void reverseTimers() {

  state = !state;

  t.oscillate(i1, 1000, state, 8);
  t.oscillate(i2, 2000, state, 4);
  t.oscillate(i3, 4000, state, 2);
  t.oscillate(i4, 8000, state, 1);

}

//// YOUR CODE ENDS HERE

// For Arduino 1.0 and earlier
// #if defined(ARDUINO) && ARDUINO >= 100
// #include "Arduino.h"
// #else
// #include "WProgram.h"
// #endif
```

```
Event::Event(void)
{
	eventType = EVENT_NONE;
}

void Event::update(void)
{
   unsigned long now = millis();
   update(now);
}

void Event::update(unsigned long now)
{
	if (now - lastEventTime >= period)
	{
		switch (eventType)
		{
			case EVENT_EVERY:
				(*callback)();
				break;

			case EVENT_OSCILLATE:
				pinState = ! pinState;
				digitalWrite(pin, pinState);
				break;
		}
		lastEventTime = now;
```

```
            count++;
        }
        if (repeatCount > -1 && count >= repeatCount)
        {
            eventType = EVENT_NONE;
        }
}


Timer::Timer(void)
{
}

int8_t Timer::every(unsigned long period, void (*callback)(), int
repeatCount)
{
        int8_t i = findFreeEventIndex();
        if (i == -1) return -1;

        _events[i].eventType = EVENT_EVERY;
        _events[i].period = period;
        _events[i].repeatCount = repeatCount;
        _events[i].callback = callback;
        _events[i].lastEventTime = millis();
        _events[i].count = 0;
        return i;
}
```

```
int8_t Timer::every(unsigned long period, void (*callback)())
{
	return every(period, callback, -1); // - means forever
}

int8_t Timer::after(unsigned long period, void (*callback)())
{
	return every(period, callback, 1);
}

int8_t Timer::oscillate(uint8_t pin, unsigned long period, uint8_t
startingValue, int repeatCount)
{
	int8_t i = findFreeEventIndex();
	if (i == NO_TIMER_AVAILABLE) return
NO_TIMER_AVAILABLE;

	_events[i].eventType = EVENT_OSCILLATE;
	_events[i].pin = pin;
	_events[i].period = period;
	_events[i].pinState = startingValue;
	digitalWrite(pin, startingValue);
	_events[i].repeatCount = repeatCount * 2; // full cycles not
transitions
	_events[i].lastEventTime = millis();
	_events[i].count = 0;
	return i;
}
```

```
int8_t Timer::oscillate(uint8_t pin, unsigned long period, uint8_t startingValue)
{
	return oscillate(pin, period, startingValue, -1); // forever
}

/**
 * This method will generate a pulse of !startingValue, occuring period after the
 * call of this method and lasting for period. The Pin will be left in !startingValue.
 */
int8_t Timer::pulse(uint8_t pin, unsigned long period, uint8_t startingValue)
{
	return oscillate(pin, period, startingValue, 1); // once
}

/**
 * This method will generate a pulse of startingValue, starting immediately and of
 * length period. The pin will be left in the !startingValue state
 */
int8_t Timer::pulseImmediate(uint8_t pin, unsigned long period, uint8_t pulseValue)
{
	int8_t id(oscillate(pin, period, pulseValue, 1));
```

```
        // now fix the repeat count
        if (id >= 0 && id < MAX_NUMBER_OF_EVENTS) {
                _events[id].repeatCount = 1;
        }
        return id;
}

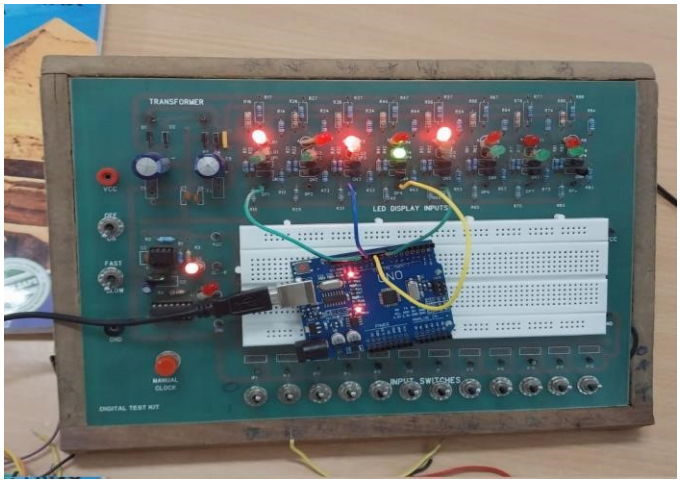

void Timer::stop(int8_t id)
{
        if (id >= 0 && id < MAX_NUMBER_OF_EVENTS) {
                _events[id].eventType = EVENT_NONE;
        }
}

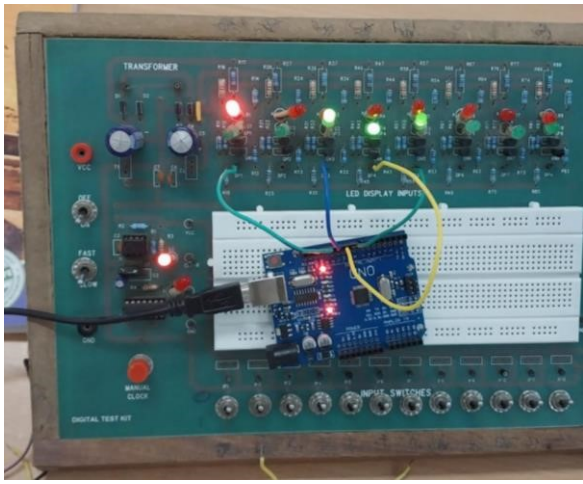
void Timer::update(void)
{
        unsigned long now = millis();
        update(now);
}

void Timer::update(unsigned long now)
{
        for (int8_t i = 0; i < MAX_NUMBER_OF_EVENTS; i++)
        {
                if (_events[i].eventType != EVENT_NONE)
                {
                        _events[i].update(now);
```

```
            }
        }
}
int8_t Timer::findFreeEventIndex(void)
{
        for (int8_t i = 0; i < MAX_NUMBER_OF_EVENTS; i++)
        {
                if (_events[i].eventType == EVENT_NONE)
                {
                        return i;
                }
        }
        return NO_TIMER_AVAILABLE;
}
```

OBSERVATION :





CONCLUSION :

We can make a counter which counts from 0 to 15 and back to 0 from 15 using Arduino.

LINK TO TINKERCAD SIMULATION :
4 BIT UP-DOWN-COUNTER